



CLASS 2 • TERRALOGIC ACADEMY • BATCH 3 (MERN STACK)

Git & Version Control

What Git is, where it came from, and how developers use it every day to track changes and collaborate safely with a team.

Securely cloning repositories • Core Git commands • Team collaboration workflow

What We'll Cover



1. What is Git & its History

Why Git was created and how it changed software development



2. How Git Works

Working directory, staging area, and the repository



3. Setup & Secure Cloning

Installing Git, configuring identity, cloning over SSH



4. Daily Workflow Commands

status, add, commit, log, diff



5. Branching & Merging

Working on features without breaking the main code



6. Collaborating With a Team

push, pull, fetch, fork, and pull requests



7. Undo & Safety Net

stash, reset, revert and recovering from mistakes



8. Cheat Sheet & Hands-on Activity

A one-page reference plus practice exercise

SECTION 1

Understanding Git

Before we touch the terminal - what problem does Git actually solve?



What is Git, and Why Do We Need It?



Git is a Version Control System

A tool that records every change made to your code over time, in a special database called a repository. It lets you:

- Save snapshots of your project (called commits)
- Go back to any earlier version instantly
- See exactly what changed, when, and who changed it
- Work on new features without breaking working code
- Let many developers edit the same project safely



Without Version Control

- project_final.js, project_final_v2.js, project_FINAL_v3.js
- Overwriting a teammate's work by accident
- No way to know who changed what, or why
- Impossible to undo a mistake cleanly



With Git

- One project folder, full history of every change
- Each person works on their own branch safely
- Every change is logged with author, time, message
- Mistakes are reversible in seconds

WHERE IT CAME FROM

The History of Git

1991-2002

Linux kernel grows

Thousands of developers email patches for the Linux kernel with no shared tool to track changes.

2005

Linus Torvalds builds Git

In about 10 days, Linus wrote a new distributed version control system, naming it 'Git'.

Today

Industry standard

Git is now the default version control system used by virtually every software team worldwide.

2002-2005

BitKeeper used, then withdrawn

The kernel team used a proprietary tool, BitKeeper. In 2005 the free-of-charge license was withdrawn.

2005-2008

Adoption grows

Junio Hamano takes over as maintainer; GitHub launches in 2008, making Git mainstream.

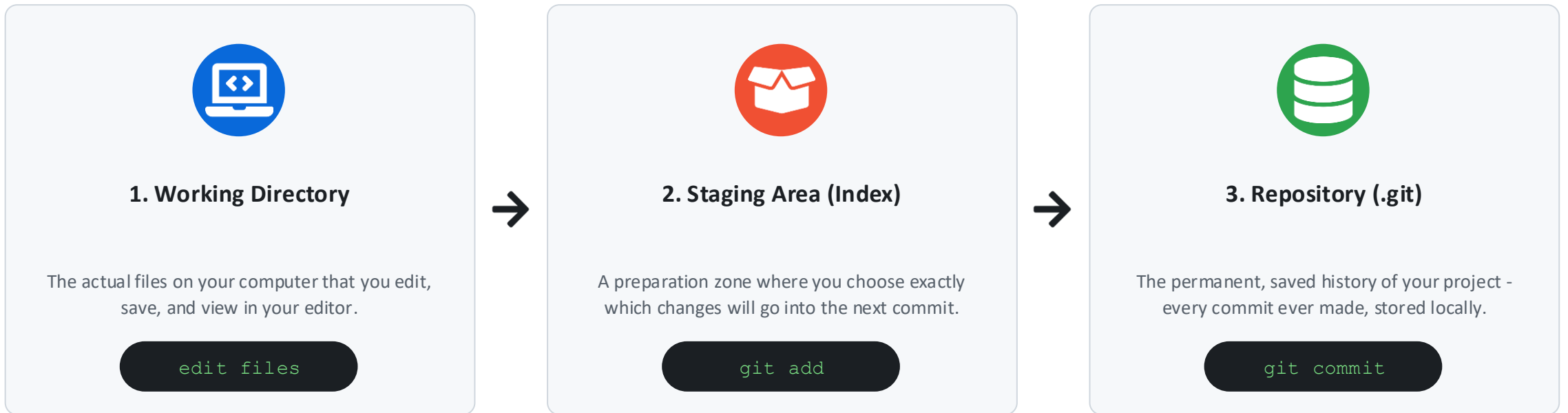
COMMON CONFUSION

Git vs. GitHub vs. Older Tools

	Git	GitHub	SVN (older tool)
What is it?	Version control software you install	A website/service that hosts Git repos online	An older, centralized version control system
Where does history live?	On every developer's own machine	In the cloud, as a backup & sharing hub	Only on one central server
Works without internet?	Yes - fully offline commits & history	No - needs internet to push/pull/view	No - needs the central server
Adds collaboration tools?	No - just the core engine	Yes - Pull Requests, Issues, Actions, Teams	Limited, server-dependent

Key takeaway: Git is the engine. GitHub is one popular garage that stores and displays what the engine produces (GitLab and Bitbucket are others).

How Git Works: The Three Areas



git push moves committed history from your local repository to a remote one (e.g. GitHub) - covered later in this session.

SECTION 2

Setting Up Git

Installing Git, configuring your identity, and starting a repository the secure way.



Installing & Configuring Git

Before your first commit, Git needs to know who you are - this name and email are stamped on every commit you make.

git --version

```
$ git --version
```

Checks whether Git is installed and which version you have.

Example: `git --version`

git config --global user.name

```
$ git config --global user.name
```

Sets the name that will be attached to all your commits, on this machine.

Example: `git config --global user.name "Satya"`

git config --global user.email

```
$ git config --global user.email
```

Sets the email attached to your commits. Should match your GitHub account email.

Example: `git config --global user.email "satya@example.com"`

git config --list

```
$ git config --list
```

Shows every configuration value currently set, so you can confirm it was saved.

Example: `git config --list`

STARTING A PROJECT

git init & git clone

There are two ways to get a Git project started on your machine: create a brand-new one, or copy an existing one.



git init

```
$ git init
```

Turns the current folder into a brand-new Git repository by creating a hidden .git folder. Use this when starting a project from scratch on your own computer.

After this, Git starts tracking the folder, but no files are tracked yet until you 'add' them.

Example: `git init my-project`



git clone <url>

```
$ git clone <repository-url>
```

Downloads a full copy of an existing repository - every file AND its entire commit history - from a remote server (like GitHub) onto your computer. This is how you join an existing team project.

A new folder is created automatically, named after the repository.

Example: `git clone git@github.com:org/smartlearn.git`

Secure Cloning With SSH Keys

Cloning and pushing code means proving who you are to GitHub. SSH keys do this without ever typing a password.

1

```
ssh-keygen -t ed25519 -C "you@email.com"
```

Generates a new SSH key pair on your machine: a private key (kept secret, never shared) and a public key.

2

```
eval "$(ssh-agent -s)" && ssh-add ~/.ssh/id_ed25519
```

Starts the SSH agent and loads your private key into it so Git can use it automatically.

3

```
cat ~/.ssh/id_ed25519.pub
```

Prints your public key so you can copy it.

4

```
Paste into GitHub - Settings - SSH and GPG keys
```

Adds your public key to your GitHub account. Now GitHub can verify it's really you.



Why this is secure: your private key never leaves your laptop and is never typed anywhere. Never share it, paste it in chat, or commit it to a repository.

Cloning Over HTTPS vs. SSH

	HTTPS	SSH (recommended)
Clone URL example	<code>https://github.com/org/repo.git</code>	<code>git@github.com:org/repo.git</code>
Authentication	Username + Personal Access Token	SSH key pair (no password typing)
Setup effort	Quick - works immediately	One-time key setup, then seamless forever
Asking for credentials?	Often prompts on every push	Never prompts once the key is added
Best for	Quick one-off clones, locked-down networks	Daily development & team collaboration

SECTION 3

Daily Git Workflow

The commands you will type dozens of times a day, in order.



STEP 1 & 2

git status & git add

Every work session follows the same loop: see what changed, then choose what to save.



git status

```
$ git status
```

Shows the current state of your working directory: which files are modified, which are new (untracked), and which are already staged for the next commit. Run this constantly - it never makes changes, only reports.

Example: *git status*



git add <file>

```
$ git add <file> | git add .
```

Moves changes from the working directory into the staging area, marking them to be included in the next commit. 'git add .' stages every changed file in the current folder at once.

Example: *git add index.js / git add .*

STEP 3

git commit



git commit -m "message"

```
$ git commit -m "Add login validation"
```

Saves a permanent snapshot of everything currently staged into the repository's history, along with a message describing the change. This is the core action of Git - every commit becomes a recoverable point in time.

Example: `git commit -m "Fix navbar overlap on mobile"`



git commit -am "..."

```
$ git commit -am "message"
```

Shortcut that stages all tracked, modified files and commits them in one step. Note: it will NOT pick up brand-new untracked files.



Writing Good Commit Messages

- Use the present tense: "Add feature" not "Added feature"
- Keep the first line under ~50 characters
- Describe WHY, not just what, when it isn't obvious
- One logical change per commit - don't bundle unrelated edits
- Never commit secrets, passwords, or API keys

Good: "Fix race condition in session refresh"

Avoid: "fixed stuff" / "wip" / "asdf"

git log & git diff

Once you have commits, you'll often need to look back - to see what happened, or to compare versions.



git log

```
$ git log --oneline --graph
```

Shows the commit history: author, date, message, and unique commit ID (hash) for each saved snapshot. The `--oneline` flag condenses each commit to one line; `--graph` visualizes branches.

Example: `git log --oneline -5`



git diff

```
$ git diff | git diff --staged
```

Shows the exact line-by-line changes between your working files and the last commit. 'git diff --staged' shows what's already in the staging area, about to be committed.

Example: `git diff src/App.js`

SECTION 4

Branching & Merging

How developers work on new features without breaking the code everyone relies on.



git branch & git switch / checkout

A branch is an independent line of work. The main branch stays stable while you experiment safely on your own.



git branch

```
$ git branch | git branch <name>
```

With no arguments, lists all branches in the repository and marks the one you're on. With a name, creates a new branch starting from your current commit - it does NOT switch to it.

Example: `git branch feature/login-page`



git switch <branch>

```
$ git switch <branch> | git switch -c <new-branch>
```

Moves you onto a different branch, updating your working files to match it. The -c flag creates AND switches to a new branch in one step. (Older tutorials use 'git checkout' for this too.)

Example: `git switch -c feature/login-page`

git merge & Resolving Conflicts



git merge <branch>

```
$ git switch main  
$ git merge feature/login-page
```

Combines the changes from another branch into your current branch. Run from the branch you want to update (usually main), naming the branch to bring in.

Example: `git merge feature/login-page`



When does a conflict happen?

A merge conflict occurs when two branches changed the SAME lines of a file differently. Git pauses the merge and asks you to manually choose which version (or a combination) to keep, then commit the result.

What a conflict looks like in a file

```
<<<<<< HEAD  
  
    const title = "Smartlearn";  
  
=====  
  
    const title = "Smartlearn ERP";  
  
>>>>>> feature/login-page
```

Edit the file to keep the correct lines, remove the markers, then run:

```
$ git add <file>  
$ git commit
```

SECTION 5

Remote Collaboration

How your local commits travel to GitHub, and how teammates' work comes back to you.



SENDING YOUR WORK

git remote & git push

A 'remote' is just a saved nickname for a repository's URL on a server like GitHub - usually called origin.



git remote -v

```
$ git remote -v | git remote add origin <url>
```

'git remote -v' lists the remote servers linked to your repo and their URLs. 'git remote add origin <url>' links your local repo to a remote for the first time (rarely needed after a clone, since clone sets this up automatically).

Example: `git remote add origin git@github.com:org/repo.git`



git push

```
$ git push origin <branch>
```

Uploads your committed history from your local branch to the matching branch on the remote (origin), making your work visible and available to the rest of the team.

Example: `git push origin feature/login-page`

RECEIVING TEAMMATES' WORK

git fetch & git pull

Your teammates are pushing commits too - these two commands bring that work down to your machine.



git fetch

```
$ git fetch origin
```

Downloads new commits and branches from the remote, but does NOT change any of your local files or branches. It's the safe way to 'check for updates' before deciding what to do next.

Example: *git fetch origin*



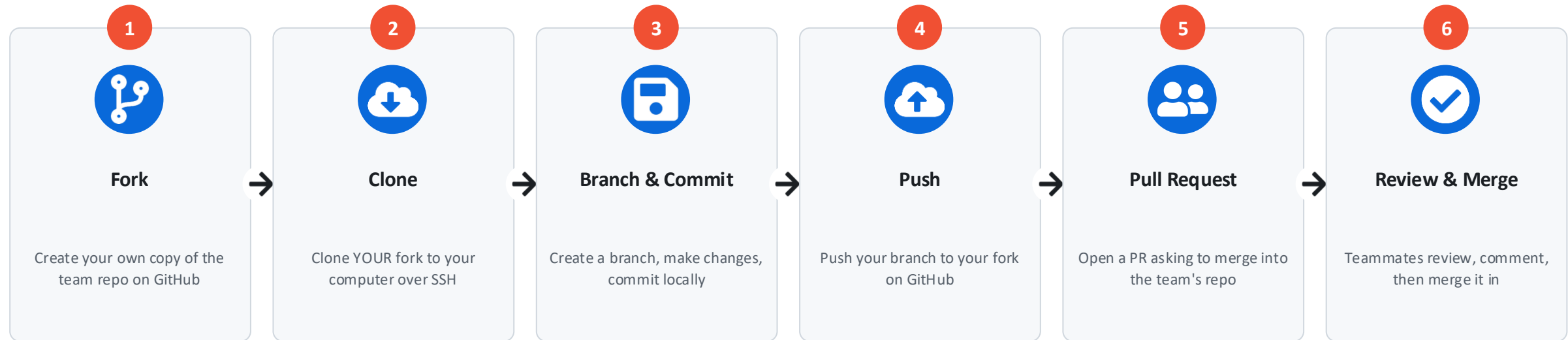
git pull

```
$ git pull origin <branch>
```

Does a 'git fetch' AND immediately merges those new commits into your current local branch in one step. This is what you run every morning before starting work, to stay in sync.

Example: *git pull origin main*

Fork & Pull Request Workflow



Why fork instead of cloning the team repo directly? Forking gives you a sandbox to experiment in without write access to the shared repository - the team reviews your Pull Request before anything touches the real codebase.

SECTION 6

Undo & Safety Net

Everyone makes mistakes. Git gives you several ways to recover cleanly.



git stash • git reset • git revert



git stash

```
$ git stash
$ git stash pop
```

Temporarily shelves uncommitted changes so you can switch branches with a clean working directory, then 'pop' them back later to continue exactly where you left off.

Example: `git stash` → `git switch main` → `git stash pop`



git reset

```
$ git reset --soft HEAD~1
$ git reset --hard HEAD~1
```

Moves your branch pointer backward. --soft keeps your changes staged for re-committing. --hard discards them completely - use with caution, it cannot be undone once gone.

Example: `git reset --soft HEAD~1`



git revert

```
$ git revert <commit-hash>
```

Creates a brand-new commit that undoes the changes from a previous commit, WITHOUT erasing history. This is the safe choice for undoing something already pushed and shared with the team.

Example: `git revert a1b2c3d`

.gitignore & Security Best Practices



.gitignore

A plain text file in your project root that tells Git which files and folders to NEVER track - so they're never added, committed, or pushed.

```
node_modules/  
.env  
*.log  
dist/  
build/  
.DS_Store  
*.pem / *.key
```

Create it with: `touch .gitignore`



Security Best Practices

- Never commit passwords, API keys, or .env files - add them to .gitignore from day one
- Use SSH keys for authentication instead of typing passwords
- Review 'git status' and 'git diff' before every commit to catch secrets early
- If a secret is committed by accident, rotate/revoke it immediately - removing it from a later commit does not remove it from history
- Keep your local Git and OS updated with security patches

The Full Git Command Cheat Sheet

Command	Category	What it does
<code>git init</code>	Setup	Create a new local repository
<code>git clone <url></code>	Setup	Copy a remote repo to your machine
<code>git config</code>	Setup	Set your name, email & preferences
<code>git status</code>	Workflow	Show changed / staged / untracked files
<code>git add <file></code>	Workflow	Stage changes for the next commit
<code>git commit -m "msg"</code>	Workflow	Save a snapshot with a message
<code>git log</code>	History	View commit history
<code>git diff</code>	History	Compare file changes line by line
<code>git branch</code>	Branching	List or create branches
<code>git switch <branch></code>	Branching	Move to a different branch
<code>git merge <branch></code>	Branching	Combine another branch into yours
<code>git remote -v</code>	Remote	List linked remote repositories
<code>git push</code>	Remote	Upload local commits to the remote
<code>git fetch</code>	Remote	Download remote changes (no merge)
<code>git pull</code>	Remote	Download AND merge remote changes
<code>git stash</code>	Undo	Temporarily shelve uncommitted work
<code>git reset</code>	Undo	Move branch pointer backward
<code>git revert <hash></code>	Undo	Undo a commit with a new commit

Hands-On Exercise: Your First Repository

Follow these steps on your own laptop. Raise your hand at any step if something doesn't look right.

1

Generate an SSH key and add the public key to your GitHub account

2

Clone the class practice repository using the SSH URL

3

Create a new branch named after your name (e.g. satya-practice)

4

Create a file called hello.txt and write one sentence about yourself

5

Run git status, then git add ., then commit with a clear message

6

Push your branch to the remote repository with git push

7

Open a Pull Request comparing your branch against main

KEY TAKEAWAYS

Git in One Slide



Git is distributed

Full history lives on your machine, not just on a server



Commit often, with clear messages

Small, well-described snapshots are easy to review and undo



Clone securely with SSH

Keys prove your identity without ever typing a password



Branch before you build

Keep main stable; experiment safely on your own branch



Push, pull, and review

Fork → branch → push → Pull Request is the standard team loop



Never commit secrets

.gitignore and good habits keep credentials out of history



Thank You

Questions? Let's open the terminal and try it together.

Terralogic Academy • Batch 3 • MERN Stack Program